COS 433: Cryptography

Announcements:

- Problem Set 4 due today

Last time:

1. Random oracle heuristic
2. RSA signatures
3. Beginning of Schnorr signatures

Schnorr/DSA Signatures

Signature scheme defined over:

- Group G of size p
  - In practice: “elliptic curve group” (ECDSA)
  - Given generator g for G
- Scheme is secure in the ROM, assuming the hardness of discrete log

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$
$\text{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x <u>but does not reveal</u> x
- Actual scheme: uses a hash function H to determine how to mask x
  - Set $R = g^r$
  - Compute $s = H(R,m)$
  - Set $\sigma = (R, s \cdot x + r)$

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$

$\text{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = H(R, m) \cdot x + r)$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Why is this secure?

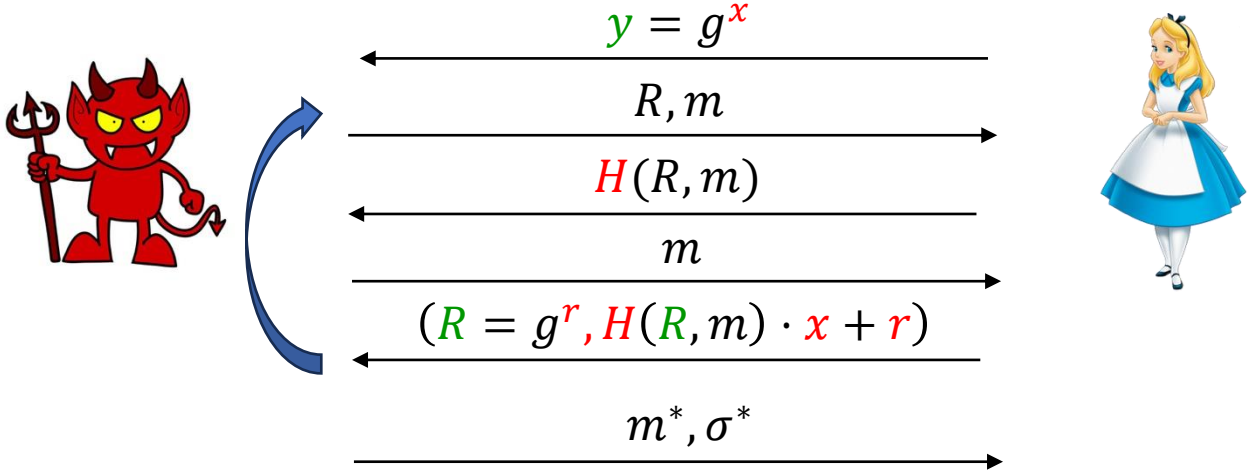
Schnorr Signatures

Key point #1: ”signing queries are useless to the adversary”

Schnorr Signatures

Key point #1: ”signing queries are useless to the adversary”

Security (random oracle model): for all efficient adversaries $A^{H(\cdot),\mathrm{Sign}(\mathrm{sk},\cdot)}$



$$\Pr_{x,H}[m^* \text{ not queried to } \mathrm{Sign}(\mathrm{sk},\cdot) \text{ and forgery}] = \mathrm{negl}(\lambda)$$

Schnorr Signatures

Key point #1: ”signing queries are useless to the adversary”

Security (random oracle model): for all efficient adversaries $A^{H(\cdot),\mathrm{Sign}(\mathrm{sk},\cdot)}$



$$y = g^x$$

$$m$$

$$(R = g^r, z = H(R,m) \cdot x + r)$$



To answer a signing query:

- Sample r
- Generate $s = H(R,m)$
- Compute $z = s \cdot x + r$

Schnorr Signatures

Key point #1: ”signing queries are useless to the adversary”

Security (random oracle model): for all efficient adversaries $A^{H(\cdot),\mathrm{Sign}(\mathrm{sk},\cdot)}$



$$y = g^x$$

$$m$$

$$(R = g^r, z = H(R,m) \cdot x + r)$$



To answer a signing query (this looks identical):

- Sample uniform z
- Sample uniform s
- Set $R = g^z \cdot y^{-s}$
- Define $H(R,m) = s$

“Signatures are not telling you anything useful”

Schnorr Signatures



$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$

$\mathrm{vk} = g^x \in G$



$m \in \{0,1\}^*$

$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\mathrm{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

0-time security (random oracle model): for all efficient adversaries $A^{H(\cdot)}$



y

R, m

$H(R,m)$

m^*, R^*, z^*



How to prove?

$$\Pr_{x,H}\left[g^{z^*} = y^{H(R^*,m^*)} \cdot R^*\right] = \mathrm{negl}(\lambda)$$

Schnorr Signatures

$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$

$\mathrm{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\mathrm{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Trivial attack:



y

m, R

$H(R,m)$

z



$$\Pr_{x,H}\left[g^z = y^{H(R,m)} \cdot R\right] = \mathrm{negl}(\lambda)$$

Schnorr Signatures

$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$
$\mathrm{vk} = g^x \in G$





$m \in \{0,1\}^*$

$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\mathrm{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Trivial attack (equivalent):



y

m, R

s

z



$$\Pr_{x,s}[g^z = y^s \cdot R] = \mathrm{negl}(\lambda)$$

Schnorr Signatures



$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$

$\mathrm{vk} = g^x \in G$



$m \in \{0,1\}^*$

$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\mathrm{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Trivial attack (equivalent):



y

R

s

z



$\Pr_{x,s}[g^z = y^s \cdot R] = \mathrm{negl}(\lambda)$

Observe: if $R = 1$ this is a discrete log attack!

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$
$\text{vk} = g^x \in G$





$y = g^x$

$(m, R = g^r, z = H(R, m) \cdot x + r)$

$m \in \{0,1\}^*$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Analysis:

- Lemma: if the scheme is secure against trivial attacks, then it is secure in the ROM
  - Proof: run  guess which (R, m) query to H gives  the forgery
  - Answer the other queries ourselves (“backwards sampling”)
- Lemma: if discrete log is hard, this scheme has no trivial attacks
  - Proof: deferred

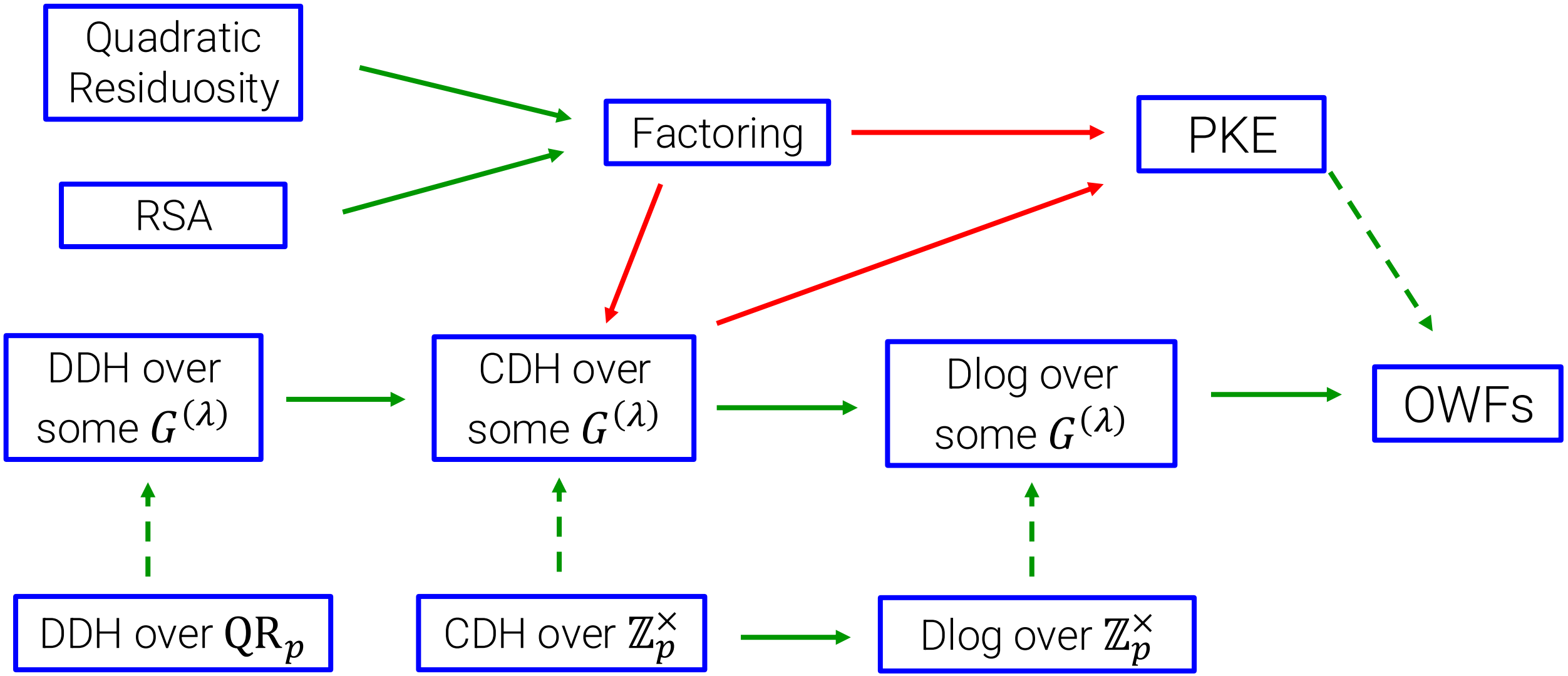
End of digital signatures!

1980s Public-Key Crypto Recap

- Public-key Encryption from number theory (RSA/QR/Factoring, Diffie-Hellman)

- Digital signatures:
  - 1-time security from CRHFs
  - Efficient many-time secure schemes from (RSA or discrete log) + random oracle

Number-Theoretic Hardness Assumptions



Part III of the course: “post-modern crypto”

Beyond Encryption + Authentication

What is cryptography for?

- Mathematically or computationally enforcing expected behavior
- Enabling trust even in the presence of untrustworthy parties

Encryption



k, m

$\mathrm{ct} = \mathrm{Enc}(k, m)$





k

$m' = \mathrm{Dec}(k, \mathrm{ct})$

- Secret communication in the presence of an untrusted eavesdropper

Authentication



k, m



$(m, \sigma = \text{Sign}(k, m))$



k

$\text{Verify}(k, m, \sigma) \in \{0, 1\}$

- Authentic communication in the presence of an (untrusted) impersonator

Beyond Encryption and Authentication





What if Alice and Bob do not trust each other?

What do they even want to accomplish?

Beyond Encryption and Authentication





Next 2-3 lectures: proof systems

Proofs

Ax. 1. $(P(\varphi) \wedge \Box\,\forall x(\varphi(x) \Rightarrow \psi(x))) \Rightarrow P(\psi)$

Ax. 2. $P(\neg\varphi) \Leftrightarrow \neg P(\varphi)$

Th. 1. $P(\varphi) \Rightarrow \Diamond\,\exists x\,\varphi(x)$

Df. 1. $G(x) \Leftrightarrow \forall\varphi(P(\varphi) \Rightarrow \varphi(x))$

Ax. 3. $P(G)$

Th. 2. $\Diamond\,\exists x\,G(x)$

Df. 2. $\varphi \text{ ess } x \Leftrightarrow \varphi(x) \wedge \forall\psi\,(\psi(x) \Rightarrow \Box\,\forall y(\varphi(y) \Rightarrow \psi(y)))$

Ax. 4. $P(\varphi) \Rightarrow \Box\,P(\varphi)$

Th. 3. $G(x) \Rightarrow G \text{ ess } x$

Df. 3. $E(x) \Leftrightarrow \forall\varphi(\varphi \text{ ess } x \Rightarrow \Box\,\exists y\,\varphi(y))$

Ax. 5. $P(E)$

Th. 4. $\Box\,\exists x\,G(x)$



Proofs

Ax. 1. $(P(\varphi) \wedge \Box\,\forall x(\varphi(x) \Rightarrow \psi(x))) \Rightarrow P(\psi)$

Ax. 2. $P(\neg\varphi) \Leftrightarrow \neg P(\varphi)$

Th. 1. $P(\varphi) \Rightarrow \Diamond\,\exists x\,\varphi(x)$

Df. 1. $G(x) \Leftrightarrow \forall\varphi(P(\varphi) \Rightarrow \varphi(x))$

Ax. 3. $P(G)$

Th. 2. $\Diamond\,\exists x\,G(x)$

Df. 2. $\varphi \text{ ess } x \Leftrightarrow \varphi(x) \wedge \forall\psi\,(\psi(x) \Rightarrow \Box\,\forall y(\varphi(y) \Rightarrow \psi(y)))$

Ax. 4. $P(\varphi) \Rightarrow \Box\,P(\varphi)$

Th. 3. $G(x) \Rightarrow G \text{ ess } x$

Df. 3. $E(x) \Leftrightarrow \forall\varphi(\varphi \text{ ess } x \Rightarrow \Box\,\exists y\,\varphi(y))$

Ax. 5. $P(E)$

Th. 4. $\Box\,\exists x\,G(x)$



Traditional/axiomatic proofs

A seminar??

Proofs



π



Prover

Verifier

CS insight: think of a proof as a form of computation

Proofs



Claim: $\forall x \forall y \forall z (\neq 0)$,
$\forall n > 2, x^n + y^n \neq z^n$

π



Prover

Verifier

CS insight: think of a proof as a form of computation

E.g.: AW writes a proof in a paper, Bob can then read the paper and check it.

Proofs



Claim: $\forall x \forall y \forall z (\neq 0)$,
$\forall n > 2, x^n + y^n \neq z^n$



π

Prover

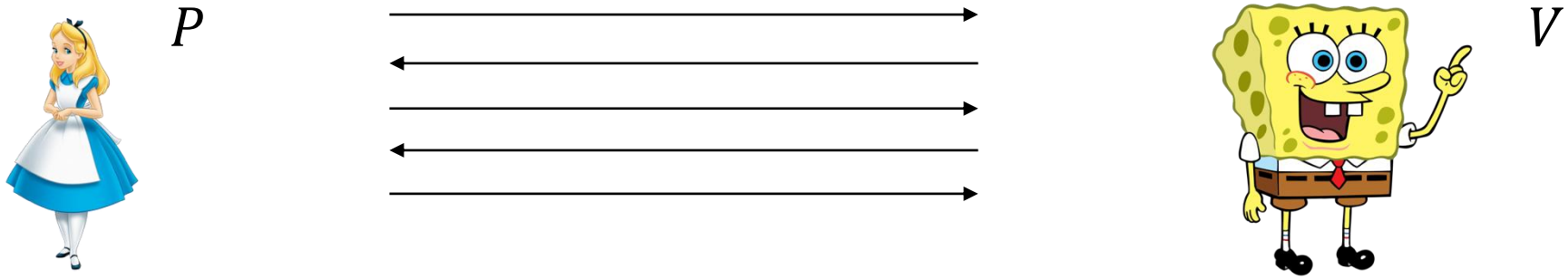
Verifier



A seminar talk:

- Does not provide an entire, formal proof
- Goal (one of many): convince the audience that a formal proof exists
- Leverages audience interaction to make the proof easier to check

Proof Systems



Definition: a proof system consists of a pair of interactive algorithms (P, V)

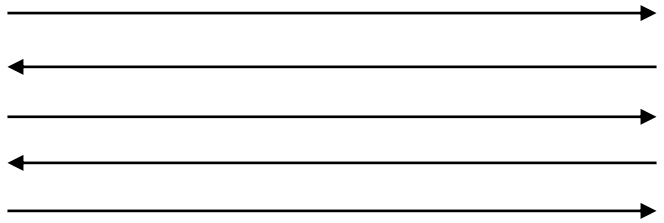
- V will always be a polynomial-time algorithm
- Polynomial with respect to what?

Proof Systems

Claim: $x \in L$



$P(x, w)$





$V(x)$

Output: $b \in \{0,1\}$

Definition: a proof system for a language $L \subset \{0,1\}^*$ consists of a pair of interactive algorithms (P, V) run on an input $x \in \{0,1\}^n$

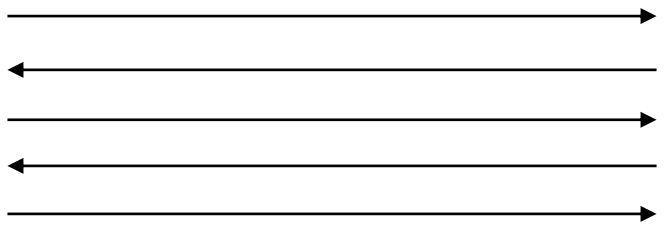
- V will always be a $\mathbf{poly}(n)$-time algorithm
- P will be either unbounded, or polynomial-time given some additional information $w \in \{0,1\}^{\mathbf{poly}(n)}$ ("efficient prover")

Proof Systems

Claim: $x \in L$



$P(x)$





$V(x)$

Output: $b \in \{0,1\}$

Definition: a proof system for a language $L \subset \{0,1\}^*$ consists of a pair of interactive algorithms (P, V) run on an input $x \in \{0,1\}^n$

- V will always be a $\mathbf{poly}(n)$-time algorithm

It must satisfy the following properties:

1. **Completeness:** if $x \in L$ and the two parties run (P, V), then V outputs 1 (accept)
2. **Soundness:** if $x \notin L$, then for any strategy P^*, V accepts with probability $\mathbf{negl}(n)$.

Note: we allow for P, V to be randomized.

Proof Systems: Examples

Claim: $N \in L$



$P(N)$ $\xrightarrow{(c,d)}$  $V(N)$

Check that $c, d > 1$ and $N = cd$

Example: composite integers

- $L = \{N : N \text{ is composite}\}$
- $P(N)$: factor $N = c \cdot d$, send (c, d)
- $V(N, c, d)$: check that $N = cd$ and $c, d > 1$

Use case: Alice could generate and release $N = pq$, and prove that it is composite at any later time by sending (p, q)

Proof Systems: Examples

Claim: $(N, y) \in L$  $P(N, y)$

x

 $V(N, y)$

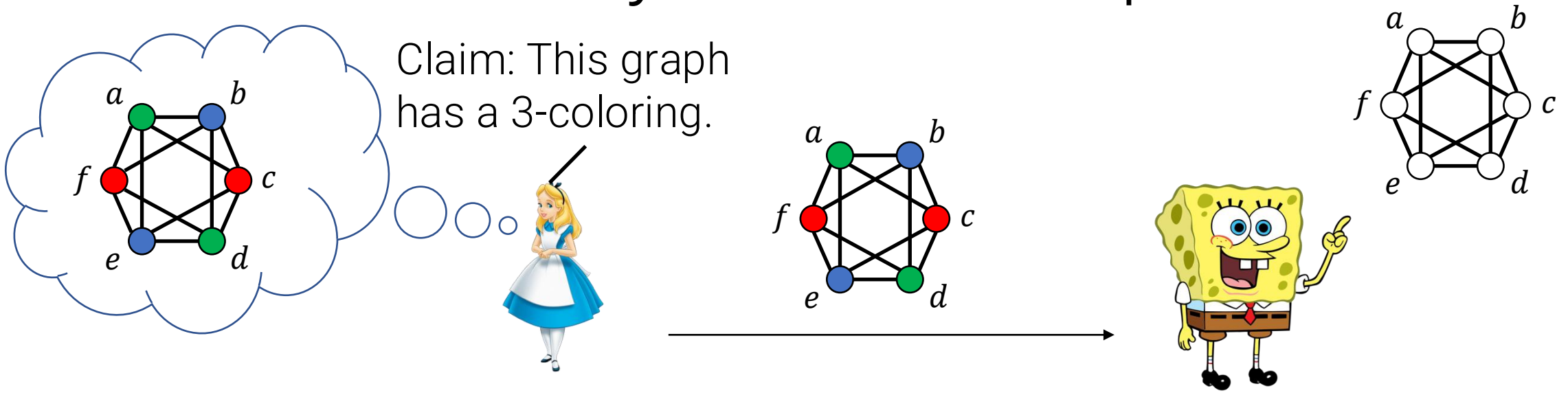
Check that $y \equiv x^2 \pmod{N}$

Example: quadratic residuosity

- $L = \{(N, y): y \text{ is a square mod } N\}$
- $P(N)$: find and send square root x
- $V(N, y, x)$: check that $y \equiv x^2 \pmod{N}$

Use case: Alice could generate and release $N = pq$, and (efficiently) prove that any square y is a perfect square (root finding is easy given factorization).

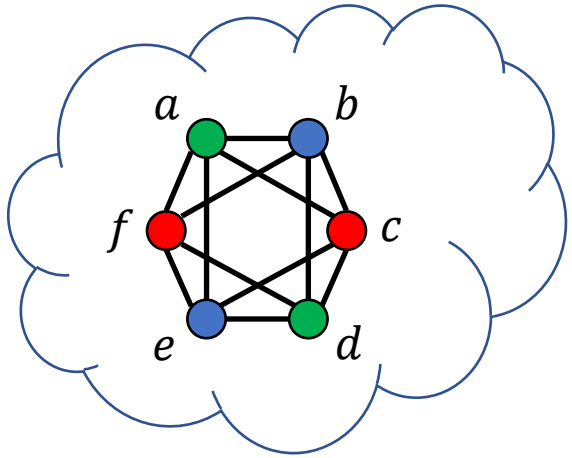
Proof Systems: Examples



Example: graph 3-coloring

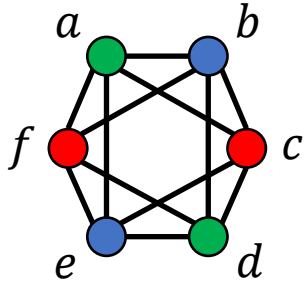
- $L = \{\text{graph } G : G \text{ is 3-colorable}\}$
- $P(G)$: find and send color assignment $c : \text{Vert}(G) \to \{0, 1, 2\}$
- $V(G, c)$: check that all edges of G are bichromatic
  - Takes time $O(n^2)$ in the worst case

Proof Systems: Examples

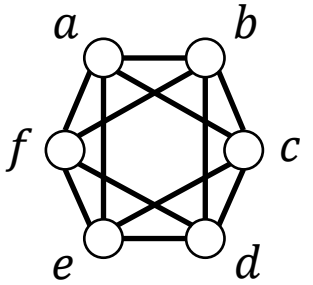


Claim: This graph has a 3-coloring.









These are all extremely simple proof systems!

- Non-interactive
- Deterministic

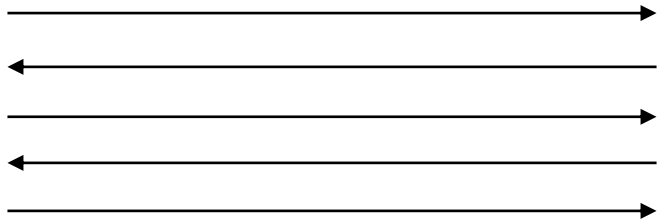
Definition: The complexity class $\mathbf{NP}$ is the set of all languages $L \subset \{0,1\}^*$ that have a deterministic, non-interactive proof system

Proof Systems: Examples

Claim: $x \in L$



$P(x, w)$





$V(x)$

Output: $b \in \{0,1\}$

Today and next time: focus on proof systems for languages $L \in \mathbf{NP}$

- For $x \in L$ and non-interactive proof system for L, call a valid proof a *witness* w
- Assume honest prover has a witness w.

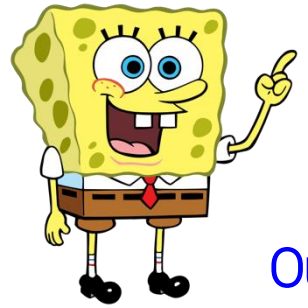
Proof Systems: Examples

Claim: $x \in L$



$P(x, w)$

w



$V(x)$

Output: $b \in \{0,1\}$

Today and next time: focus on proof systems for languages $L \in \mathbf{NP}$

- For $x \in L$ and non-interactive proof system for L, call a valid proof a *witness* w
- Assume honest prover has a witness w.

- Trivial proof system: P sends w, V checks that w is valid.

- We've done nothing…?

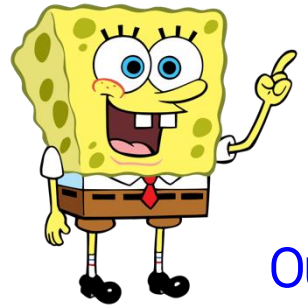
Proof Systems

Claim: $x \in L$



$P(x, w)$

w



$V(x)$

Output: $b \in \{0,1\}$

Where is the cryptography?

Idea: maybe w is supposed to be a secret!

- w could be Alice's secret key, x could be an associated public key
- w could contain sensitive data that Alice doesn't want to (or legally cannot) release

Proof Systems: Quadratic Non-Residuosity

Claim: $y \notin \mathrm{QR}_N$



$P(p, q, y)$

p, q



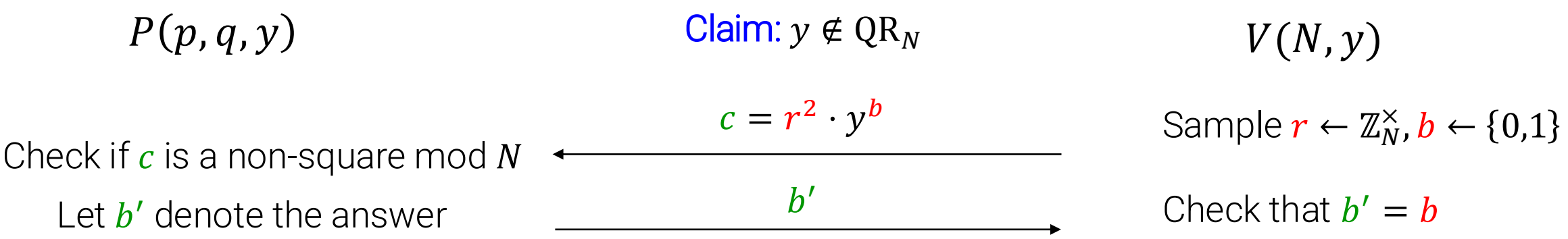
$V(N, y)$

Check $N = pq$, check if y is a square.

Can we prove y is not a square, without revealing the factorization of N?

- Problem: verifier will not be able to decide if $y \in \mathrm{QR}_N$ on his own.
- Solution: leverage **interaction** and **randomness**

Proof Systems: Quadratic Non-Residuosity



- **Completeness**: if y is not a square, then c is a non-square if and only if $b = 1$
- **Soundness**: if y is a square,
  - c is a random square mod N
  - so c hides the value of b
  - P^* can only output $b' = b$ with probability ½
  - "Soundness error ½"

Proof Systems: Quadratic Non-Residuosity

$P(p, q, y)$ — Claim: $y \notin \mathrm{QR}_N$ — $V(N, y)$

Check if c_i is a non-square mod N

Let b_i' denote the answer

$(c_i = r^2 \cdot y^b)_i$ ←

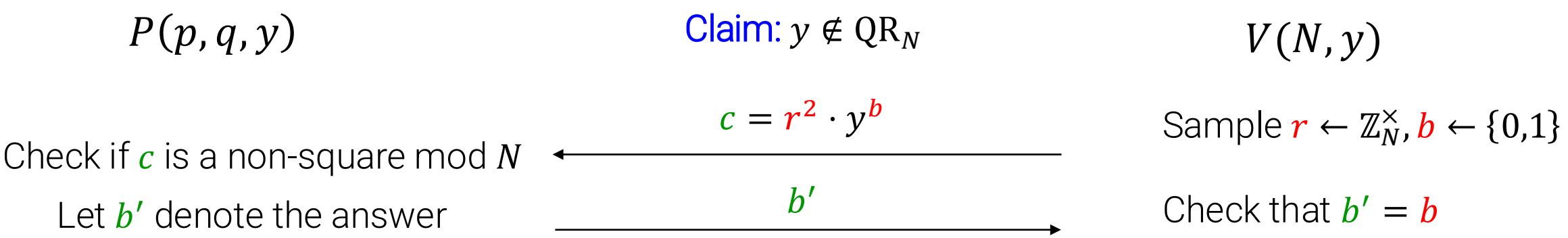
$(b_i')_i$ →

Sample $r_1, \ldots, r_n, b_1, \ldots, b_n$

Check that $b_i' = b_i$ for all i

- **Soundness**: P^* can only output all $b_i' = b_i$ with probability 2^{-n}.

Proof Systems: Quadratic Non-Residuosity



Question: can V learn the factorization of N after receiving a proof?

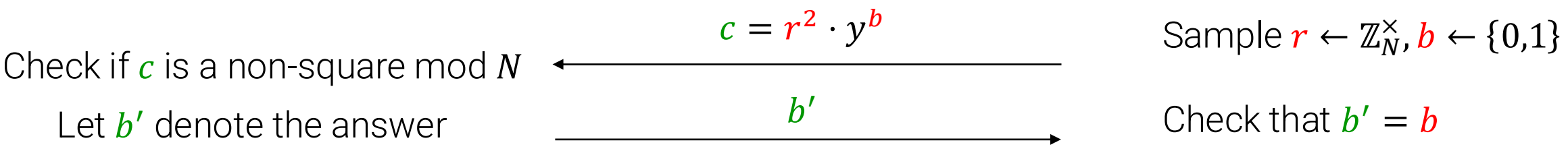
- Let's assume that V follows the protocol

Answer: no (assuming factoring N is hard), and we can prove this!

- Suppose V could output (p, q) additionally given (r, b, c, b')
- Algorithm $V(N, y, r, b, c, b') \rightarrow (p, q)$
- Then we can efficiently factor N: run $V(N, y, r, b, c, b)$

Proof Systems: Quadratic Non-Residuosity

$P(p, q, y)$ | Claim: $y \notin \mathrm{QR}_N$ | $V(N, y)$



Amazing property: V seems to learn **nothing** by receiving a proof!

- V does learn that y is a non-square
- But that's it!

Proof Systems: Quadratic Non-Residuosity

$P(p, q, y)$ | Claim: $y \notin \mathrm{QR}_N$ | $V(N, y)$

$c = r^2 \cdot y^b$ →

Check if c is a non-square mod N

Let b' denote the answer

Sample $r \leftarrow \mathbb{Z}_N^\times, b \leftarrow \{0,1\}$

b' →

Check that $b' = b$

We formalize this via a security property called (Honest-Verifier) Zero Knowledge